AI Assistant Coding

Assignment 18.1

Name : V.HANSIKA **HT. No :** 2303A52409 **Batch:** 32

Task 1 – Weather API Integration

Connect to a Weather API to fetch current weather details for a given city.

Instructions:

- Prompt AI to generate Python code using the requests library.
- Handle errors: invalid city name, API key errors, and network failures.
- Display temperature, humidity, and weather description in a readable format.

Code:

```
'''
weather feature using open-meteo API and geocoding API to get weather data
based on user input location.
'''
import requests


def get_weather(location):
    location = location.strip()
    if len(location) < 3:
        return "Please enter a valid location name with at least 3
characters."

    # Geocoding API to get latitude and longitude from location
    geocoding_url = 'https://nominatim.openstreetmap.org/search'
    geocoding_params = {
        'q': location,
        'format': 'json',
        'limit': 1,
    }
    geocoding_headers = {
        # Nominatim requires a valid User-Agent.
        'User-Agent': 'weather-cli/1.0 (student-project)'
    }
```

```
    try:
        geocoding_response = requests.get(
            geocoding_url,
            params=geocoding_params,
            headers=geocoding_headers,
            timeout=10,
        )
        geocoding_response.raise_for_status()
        geocoding_data = geocoding_response.json()
    except requests.exceptions.RequestException as exc:
        return f"Unable to fetch location data: {exc}"
    except ValueError:
        return "Location service returned an invalid response. Please try
again later."

    if not geocoding_data:
        return "Location not found. Please try again."

    latitude = geocoding_data[0]['lat']
    longitude = geocoding_data[0]['lon']

    # Open-Meteo API to get weather data
    weather_url = 'https://api.open-meteo.com/v1/forecast'
    weather_params = {
        'latitude': latitude,
        'longitude': longitude,
        'current': 'temperature_2m,relative_humidity_2m,wind_speed_10m',
    }

    try:
        weather_response = requests.get(weather_url, params=weather_params,
timeout=10)
        weather_response.raise_for_status()
        weather_data = weather_response.json()
    except requests.exceptions.RequestException as exc:
        return f"Unable to fetch weather data: {exc}"
    except ValueError:
        return "Weather service returned an invalid response. Please try again
later."

    if 'current' in weather_data:
        current = weather_data['current']
        temperature = current.get('temperature_2m')
        windspeed = current.get('wind_speed_10m')
        humidity = current.get('relative_humidity_2m')

        if temperature is None or windspeed is None:
```

```
            return "Weather data is incomplete for this location. Please try
again later."

        if humidity is None:
            return f"The current temperature in {location} is {temperature}°C
with a wind speed of {windspeed} km/h. Humidity data is unavailable."

        return f"The current temperature in {location} is {temperature}°C with
a wind speed of {windspeed} km/h and humidity of {humidity}%."
    else:
        return "Weather data not available for this location."
# Example usage
if __name__ == "__main__":
    location = input("Enter a location to get the current weather: ")
    print(get_weather(location))
```

Output:

```
(AIAC_env) PS D:\Course\AIAC\labapi253> python weather.py
# Enter a location to get the current weather: warangal
# The current temperature in warangal is 29.1°C with a wind speed of 3.3 km/h
and humidity of 62%.
# (AIAC_env) PS D:\Course\AIAC\labapi253> python weather.py
# Enter a location to get the current weather: SR University
# The current temperature in SR University is 25.9°C with a wind speed of 1.8
km/h and humidity of 79%.
# (AIAC_env) PS D:\Course\AIAC\labapi253> python weather.py
# Enter a location to get the current weather: Devannapet
# The current temperature in Devannapet is 27.7°C with a wind speed of 2.3
km/h and humidity of 69%.
```

Task 2 – Currency Exchange Rates

Integrate a Currency Exchange API to convert INR to USD, EUR, and GBP.

Instructions:

- Use AI to generate API request code.
- Validate user input for amount and currency codes.
- Implement error handling for invalid responses and API downtime.
- Show conversion results clearly in tabular format.

Code:

```
'''
desigen a currency converter that takes in the amount, the source currency,
and the target currency, and returns the converted amount.
The converter should support at least 5 different currencies and use real-time
exchange rates from an API.
use free api like exchangerate-api.com or openexchangerates.org to get the
exchange rates.
Handle errors such as invalid currency codes, missing API key, and request
failures.
'''
import os
import requests

API_KEY = os.getenv("4b520c31e5290f6c6d7887d0")
API_URL = "https://api.exchangerate-api.com/v4/latest/"
VALID_CURRENCIES = {"USD", "EUR", "GBP", "JPY", "AUD","INR"}

def fetch_exchange_rates(base_currency):
    try:
        response = requests.get(f"{API_URL}{base_currency}", timeout=10)
        response.raise_for_status()
        data = response.json()
        return data.get("rates", {}), None
    except requests.exceptions.RequestException as exc:
        return None, f"Error fetching exchange rates: {exc}"

def convert_currency(amount, source_currency, target_currency):
    source_currency = source_currency.strip().upper()
    target_currency = target_currency.strip().upper()

    if source_currency not in VALID_CURRENCIES:
        return f"Invalid source currency code. Supported currencies: {',
'.join(VALID_CURRENCIES)}"

    if target_currency not in VALID_CURRENCIES:
        return f"Invalid target currency code. Supported currencies: {',
'.join(VALID_CURRENCIES)}"

    exchange_rates, error = fetch_exchange_rates(source_currency)
    if error:
        return error

    if target_currency not in exchange_rates:
        return f"Exchange rate for {target_currency} not found."

    try:
```

```
        converted_amount = amount * exchange_rates[target_currency]
        return f"{amount} {source_currency} is equal to {converted_amount:.2f}
{target_currency}"
    except Exception as exc:
        return f"Error during conversion: {exc}"

# Example usage:
if __name__ == "__main__":
    amount = int(input("Enter the amount to convert: "))
    source_currency = input("Enter the source currency code (e.g., USD): ")
    target_currency = input("Enter the target currency code (e.g., EUR): ")
    result = convert_currency(amount, source_currency, target_currency)
    print(result)
```

Output:

```
(AIAC_env) PS D:\Course\AIAC\labapi253> python currencyConvert.py
Enter the amount to convert: 100
Enter the source currency code (e.g., USD): USD
Enter the target currency code (e.g., EUR): INR
100 USD is equal to 9405.00 INR
(AIAC_env) PS D:\Course\AIAC\labapi253> python currencyConvert.py
Enter the amount to convert: 100
Enter the source currency code (e.g., USD): INR
Enter the target currency code (e.g., EUR): USD
100 INR is equal to 1.06 USD
(AIAC_env) PS D:\Course\AIAC\labapi253> python currencyConvert.py
Enter the amount to convert: 100
Enter the source currency code (e.g., USD): INR
Enter the target currency code (e.g., EUR): EUR
100 INR is equal to 0.92 EUR
(AIAC_env) PS D:\Course\AIAC\labapi253>
```

Task 3 – GitHub Repository Info Fetcher

Connect to the GitHub API to fetch details of a repository (e.g., stars, forks, issues).

Instructions:

- Prompt AI to write a Python function to send GET requests to GitHub API.
- Handle rate limit errors, 404 (repo not found), and invalid input.
- Display repository name, description, stars, forks, and open issues.

Code:

```
'''
GitHub Repository Info Fetcher using GitHub REST API.
Fetches repository details including stars, forks, description, and open
issues.
'''
import requests


def get_github_repo_info(owner, repo_name):
    """
    Fetch GitHub repository information from the GitHub REST API.

    Args:
        owner (str): GitHub repository owner username.
        repo_name (str): GitHub repository name.

    Returns:
        dict: Repository information or error message.
    """
    # Input validation
    owner = owner.strip() if owner else ""
    repo_name = repo_name.strip() if repo_name else ""

    if not owner or not repo_name:
        return "Error: Please provide both owner and repository name."

    if len(owner) < 1 or len(repo_name) < 1:
        return "Error: Owner and repository name must be at least 1 character
long."

    # GitHub API endpoint
    api_url = f"https://api.github.com/repos/{owner}/{repo_name}"

    headers = {
        'Accept': 'application/vnd.github.v3+json',
        'User-Agent': 'GitHub-Repo-Fetcher/1.0'
    }

    try:
        response = requests.get(api_url, headers=headers, timeout=10)

        # Handle rate limit
        if response.status_code == 403:
            remaining = response.headers.get('X-RateLimit-Remaining',
'Unknown')
            reset_time = response.headers.get('X-RateLimit-Reset', 'Unknown')
```

```
            return f"Error: GitHub API rate limit exceeded. Remaining
requests: {remaining}. Reset time: {reset_time}"

        # Handle not found
        if response.status_code == 404:
            return f"Error: Repository '{owner}/{repo_name}' not found. Please
check the owner and repository name."

        # Handle other HTTP errors
        if response.status_code != 200:
            return f"Error: Failed to fetch repository data. HTTP Status Code:
{response.status_code}"

        # Parse JSON response
        try:
            repo_data = response.json()
        except ValueError:
            return "Error: GitHub API returned an invalid response. Please try
again later."

        # Extract required fields with safe access
        repo_info = {
            'name': repo_data.get('name', 'N/A'),
            'description': repo_data.get('description') or 'No description
available',
            'stars': repo_data.get('stargazers_count', 0),
            'forks': repo_data.get('forks_count', 0),
            'open_issues': repo_data.get('open_issues_count', 0),
            'url': repo_data.get('html_url', 'N/A'),
        }

        return repo_info

    except requests.exceptions.Timeout:
        return "Error: Request timed out. GitHub API is not responding."
    except requests.exceptions.ConnectionError:
        return "Error: Network connection error. Please check your internet
connection."
    except requests.exceptions.RequestException as exc:
        return f"Error: Failed to fetch repository data: {exc}"


def display_repo_info(repo_info):
    """
    Display repository information in a formatted manner.

    Args:
        repo_info (dict or str): Repository information or error message.
    """
```

```python
    if isinstance(repo_info, str):
        # Error message
        print(repo_info)
    else:
        # Success - display repository details
        print("\n" + "="*60)
        print("GitHub Repository Information")
        print("="*60)
        print(f"Repository Name: {repo_info['name']}")
        print(f"URL: {repo_info['url']}")
        print(f"Description: {repo_info['description']}")
        print(f"Stars: {repo_info['stars']:,}")
        print(f"Forks: {repo_info['forks']:,}")
        print(f"Open Issues: {repo_info['open_issues']:,}")
        print("="*60 + "\n")


# Main execution
if __name__ == "__main__":
    print("GitHub Repository Info Fetcher")
    print("-" * 40)

    owner = input("Enter GitHub repository owner (username): ").strip()
    repo_name = input("Enter GitHub repository name: ").strip()

    print("\nFetching repository information...")
    repo_info = get_github_repo_info(owner, repo_name)
    display_repo_info(repo_info)
```

Output:

```
(AIAC_env) PS D:\Course\AIAC\labapi253> python githunInfoFeature.py
GitHub Repository Info Fetcher
----------------------------------------
Enter GitHub repository owner (username): Eshwar1435
Enter GitHub repository name: AIAC-LAB-2072

Fetching repository information...

============================================================
GitHub Repository Information
============================================================
Repository Name: AIAC-LAB-2072
URL: https://github.com/Eshwar1435/AIAC-LAB-2072
Description: No description available
Stars: 0
Forks: 0
Open Issues: 0
============================================================
```

Task 4 – Real-Time Application: News Headlines Aggregator

Scenario: Build a News Aggregator using a free News API.

Requirements:

- Fetch top 5 headlines for a given category (sports, technology, health).
- Handle errors like invalid category, missing API key, and request failures.
- Display headlines in a user-friendly numbered list format.
- Include a retry mechanism if the first request fails.

Code:

```
'''
Build a News Aggregator  using newsapi.org API to fetch and display news
headlines based on user input topic or category.
Fetch top 5 headlines for a given category (sports, technology,
health). Handle errors like invalid category, missing API key, and request
failures. Display headlines in a user-friendly numbered list format. Include a
retry mechanism if the first request fails.
'''
import os
import time

import requests

API_KEY = os.getenv("NEWS_API_KEY", "6a64706bdc3746f9ba16dda387329610")
BASE_URL = "https://newsapi.org/v2/top-headlines"
VALID_CATEGORIES = {"sports", "technology", "health"}


def fetch_with_retry(params, retries=1, timeout=10):
    last_error = None

    for attempt in range(retries + 1):
        try:
            response = requests.get(BASE_URL, params=params, timeout=timeout)
            response.raise_for_status()
            return response.json(), None
        except requests.exceptions.RequestException as exc:
            last_error = exc
            if attempt < retries:
```

```
                time.sleep(1)

    return None, f"Unable to fetch news headlines after {retries + 1}
attempt(s): {last_error}"

def get_news_headlines(category):
    category = category.strip().lower()

    if category not in VALID_CATEGORIES:
        return "Invalid category. Please choose from: sports, technology,
health."

    if not API_KEY or API_KEY == "YOUR_API_KEY_HERE":
        return "Missing API key. Set NEWS_API_KEY environment variable or
update API_KEY in the script."

    params = {
        "category": category,
        "pageSize": 5,
        "country": "us",
        "apiKey": API_KEY
    }

    data, request_error = fetch_with_retry(params, retries=1, timeout=10)
    if request_error:
        return request_error

    if not isinstance(data, dict):
        return "News service returned an invalid response. Please try again
later."

    if data.get("status") != "ok":
        return f"Error fetching news headlines: {data.get('message', 'Unknown
error')}"

    headlines = data.get("articles", [])
    if not headlines:
        return f"No headlines found for category: {category}"

    result_lines = [f"Top 5 Headlines for {category.capitalize()}:"]
    for i, article in enumerate(headlines, start=1):
        title = article.get("title") or "Untitled headline"
        result_lines.append(f"{i}. {title}")

    return "\n".join(result_lines)

if __name__ == "__main__":
    category = input("Enter a category (sports, technology, health): ")
```

```
    print(get_news_headlines(category))
```

Output:

```
(AIAC_env) PS D:\Course\AIAC\labapi253> python newsHeadline.py
Enter a category (sports, technology, health): sports
Top 5 Headlines for Sports:
1. MLB 2026 season preview: Rankings, every team's playoff odds - ESPN
2. Syracuse finalizes deal with Gerry McNamara to become next head coach: ‘I’m
ready for it’ - Syracuse.com
3. UK booster asks President Capilouto to reverse Mitch Barnhart’s $950K role
- On3
4. Post-Free Agency NFL Power Rankings: Bengals, Commanders Are Contenders
Again - Sports Illustrated
5. 'If he'd known what condition the bike was in, he wouldn't have been
descending like that' – Tadej Pogačar won Milan-San Remo on a cracked frame
with disc brake rub - Cyclingnews
(AIAC_env) PS D:\Course\AIAC\labapi253> python newsHeadline.py
Enter a category (sports, technology, health): technology
Top 5 Headlines for Technology:
1. Just get a new Samsung phone? Turn off these 6 features immediately -
Android Authority
2. We Didn't Expect This Minecraft Spin-Off to Get a PS5 Sequel - Push Square
3. Nintendo Cuts Switch 2 Output by Over 30% on Weak US Sales - Bloomberg.com
4. Android Auto is breaking for Pixel and Samsung users, and no one knows why
- Android Authority
5. Claude can now automate your entire desktop, but with a serious limitation
- XDA
(AIAC_env) PS D:\Course\AIAC\labapi253>
```